

Why pushing the strong simplifier through derivatives is not free

Symbols. Regexes r, t over Σ : $\mathbf{0}$, $\mathbf{1}$, c ($c \in \Sigma$), $r \cdot t$, $r + t$, r^* . Derivative ∂_c and word-derivative ∂_s :

$$\begin{aligned} \partial_c c &= \mathbf{1}, & \partial_c d &= \mathbf{0} \ (d \neq c), & \partial_c \mathbf{1} &= \partial_c \mathbf{0} = \mathbf{0}, & \partial_c(r+t) &= \partial_c r + \partial_c t, \\ \partial_c(r \cdot t) &= \partial_c r \cdot t + [\nu r] \partial_c t, & \partial_c(r^*) &= \partial_c r \cdot r^*, & \partial_\varepsilon &= \text{id}, & \partial_{cs} &= \partial_s \circ \partial_c, \end{aligned}$$

with $[\nu r]$ present iff r is nullable, and size $\|\cdot\|$.

Two simplifiers:

w : the *verified weak* simplifier — the one the proven-correct lexer iterates;
 S : the *strong* simplifier whose universe the cubic bound measures.

S is strictly stronger than w : on top of w 's rules it elides units $\mathbf{1} \cdot k \mapsto k$, **collapses adjacent equal stars** $a^* \cdot a^* \mapsto a^*$, and prunes alternation branches *set-wise*. S preserves the *language* but *not* the POSIX value.

The real object vs. the measured object. The verified lexer's regex after reading $s = c_1 \cdots c_n$ is

$$D_s(r) = (w \circ \partial_{c_n}) \circ \cdots \circ (w \circ \partial_{c_1})(r).$$

The cubic bound instead measures the *strong* object $S(\partial_s r)$ (abstracting the row-set it really bounds). To make the bound apply to the real lexer one needs the one-directional size domination

$$\boxed{\|D_s(r)\| \leq \|S(\partial_s r)\| \quad \text{for all } r, s.} \quad (\star)$$

The only cheap proof of $(\star)$ = a one-step simulation. Induct on s , pushing S past one derivative at a time. Since $D_{cs}(r) = D_s(w(\partial_c r))$ while $S(\partial_{cs} r) = S(\partial_c(\partial_s r))$, the induction closes *iff* S almost commutes with ∂_c :

$$\partial_c(Sr) \rightsquigarrow^* S(\partial_c r) \quad (\text{C})$$

(∂ of a strong normal form rewrites, in the verified order $\rightsquigarrow$, down to the strong normal form of the ∂).

(C) is false already at the smallest term. Let $r = b \cdot c$ with $b \neq c$, and differentiate by b :

$$\begin{aligned} Sr &= b \cdot c, & \partial_b(Sr) &= \partial_b(b \cdot c) = \mathbf{1} \cdot c, \\ \partial_b r &= \mathbf{1} \cdot c, & S(\partial_b r) &= S(\mathbf{1} \cdot c) = c. \end{aligned}$$

Hence

$$\partial_b(Sr) = \mathbf{1} \cdot c \neq c = S(\partial_b r),$$

and the near-identity step relation $\rightsquigarrow$ has no rule $\mathbf{1} \cdot k \rightsquigarrow k$ at this position, so $\mathbf{1} \cdot c \not\rightsquigarrow^* c$. Thus (C) fails. (*This is the documented minimal refutation of `rprime_step`'s commutation.*)

Why it cannot be cheaply patched. ∂_c does *not* preserve strong normal forms: $\partial_b(b \cdot c) = \mathbf{1} \cdot c$ is not S -normal. Re-normalising after each ∂ requires the *full* S again, not a local step $\rightsquigarrow$. Closing $\rightsquigarrow$ under full S -normalisation makes it the entire verified rewrite system *on top of* S — i.e. re-proving the strong simplifier correct through derivatives, which *is* the original problem, not a free lemma.

The deeper instance — the $a^* a^*$ wall. S 's signature rule $a^* \cdot a^* \mapsto a^*$ fires only at a sequence *top*. ∂_c splits sequences and re-exposes buried subterms, so it *moves the redex*: one side collapses an $a^* a^*$ that the other leaves buried under a leading factor. This is the same collapse asymmetry that has defeated every strong-vs-weak inclusion in this project. Therefore even $(\star)$ — which needs only a one-directional *inequality*, not (C) as an equality — is not free, and must be falsification-tested before further work rests on it.